RISK-BASED FRAMEWORK FOR ADAPTIVE QUALITY OF SERVICE AND THREAT-AWARE RESPONSE IN SOFTWARE-DEFINED NETWORKING

25-26J-028

Project Proposal Report

Perera K.C.S.P

B.Sc. (Hons) in Information Technology Specializing in Computer Systems and Network Engineering

Department of Computer Systems Engineering

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

Declaration

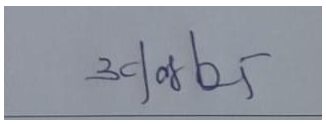
To the best of my knowledge and belief, this proposal does not contain any previously published or written by another person material, except where the acknowledgement is made in the text. I declare that this is solely my own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning.

Group Member Name	Student ID	Signature
Perera K.C.S.P	IT22324306	

The above candidates are carrying out research for their undergraduate dissertations under my supervision.

Name of supervisor: Mr. Uditha Dharmakeerthi

Name of co-supervisor: Ms. Shashika Lokuliyana

Name	Signature	Date
Mr. Uditha Dharmakeerthi		
Ms. Shashika Lokuliyana		

ABSTRACT

This research focuses on the development of a Risk-Based Framework for adaptive Quality of Service (QoS) and threat-aware responses in Software-Defined Networking (SDN) environments. SDN, while offering flexibility and scalability, introduces vulnerabilities, particularly within the centralized controller. Traditional Intrusion Detection Systems (IDS) based on binary classifications fail to account for the severity of threats, leading to false positives and inadequate responses. To address this, the research proposes the development of a Threat Evaluation Component that assigns dynamic threat scores to incoming network flows, enabling real-time, context-aware decision-making. The system integrates machine learning models with SDN, leveraging benchmark datasets (CICIDS2018) to identify and score threats accurately. This research aims to enhance SDN security by moving beyond binary alerts, improving scalability, and enabling adaptive policy enforcement based on severity-based threat evaluation.

Keywords: SDN, Threat Detection, QoS, Threat Scoring, Adaptive Policy Engine, Machine Learning, Intrusion Detection, Network Security.

Table of Contents

Declaration i

ABSTRACT ii

INTRODUCTION 1

RESEARCH GAP 2

RESEARCH PROBLEM 4

OBJECTIVES 4

Main Objectives 4

Specific Objectives 5

METHODOLOGY 5

Overall System Description 5

PROJECT REQUIREMENTS 9

REFERENCES 14

INTRODUCTION

Background study

Software-Defined Networking (SDN) is now a fundamental technology enabling the development of programmable, flexible and centrally controlled networks. SDN allows the installation of rules dynamically and provides visibility of the traffic globally by isolating the control plane with the data plane. Nevertheless, this architecture also presents uncommon vulnerabilities. The SDN controller becomes a single point of failure, as well as a very appealing attack target. DDoS/Denial-of-Service (DoS), brute force can overload the data plane and cause service degradation or failure. Intrusion Detection Systems (IDS) traditionally are based on signature-based methods and are ineffective with zero-day or evolving attacks. To address that, scholars have started focusing their investigations on the use of Machine Learning (ML) and Deep Learning (DL) in anomaly detection in SDN traffic. Such datasets as CICIDS2018 have been adopted as reference datasets to train such models. ML baselines such as the Random Forests and SVM are quick but not as accurate, whereas DL baselines (e.g., CNNs, LSTMs, hybrid DNN-LSTM) are able to learn complicated temporal-spatial traffic patterns and are systematically better than ML baselines. In spite of such developments, the majority of current IDS within the SDN are based on a two-category classification paradigm: traffic can be deemed as either benign or malicious. Others of these frameworks are the multi-classification, which defines various forms of attacks. Nevertheless, such methods do not take into consideration the severity of attacks. A harmless flow false-labeled as malicious could be blocked without need, damaging actual services, and a small port scan and a volumetric DDoS attack are both reported with identical results in detection output. This cannot distinguish severity, which compromises the adaptive capabilities of SDN.

Literature survey

Several notable works have advanced SDN-based intrusion detection, but all share common limitations that motivate the **Threat Score Module**:

- **Network Threat Detection using ML/DL (2020):** A comparative analysis of the ML and DL methods on intrusion detection in benchmark datasets was conducted. CNN and LSTM DL models were more accurate (~9899), and had less false positives, but had discrete threat scores. [Network Threat Detection on ML/DL, 2020].
- **Cyber Threats Detection in Smart Environments using SDN-enabled DNN-LSTM (2021):** Proposed a hybrid CuDNN-LSTM model embedded in SDN controllers, trained on CICIDS2018, achieving ~99.5% accuracy with very low false positives. Despite its strong results, the system still only labeled traffic as benign/malicious or into fixed attack classes, with no mechanism to assign **severity levels**. [Cyber Threats Detection in SDN, 2021]

- **Deep Learning IDS for SDN (2022):** Designed a DNN-based IDS for SDN, proving deep learning outperforms traditional ML in SDN contexts. However, the evaluation was limited to classification accuracy and did not explore QoS preservation or severity-based decision-making. [Deep Learning IDS in SDN, 2022]

RESEARCH GAP

Although SDN-based IDS using ML/DL have achieved high accuracy in identifying attacks, several gaps remain that prevent them from being adopted as **practical, risk-aware threat scoring mechanisms**. Existing approaches primarily offer classification, with limited attention to **threat severity quantification, score calibration, or real-time integration into SDN controllers**. The following table summarizes the major gaps identified from prior works and how the proposed Threat Score Module addresses them.

Feature / Focus Area	**Current Research**	**Identified Gap**	**Proposed Solution (This Research)**
Threat Detection in SDN	Most studies use binary intrusion detection (benign vs. malicious) with ML/DL techniques on datasets like CICIDS2017/2018.	Oversimplifies cyber threats, ignores severity differences; results in rigid responses that either block or allow traffic.	Develop a **Threat Evaluation Component** that assigns a continuous threat score (e.g., 0–10) to traffic flows, reflecting severity and impact.
Machine Learning Approaches	Lack mechanisms to distinguish between malicious traffic and unusual but legitimate behaviors (e.g., backups, software updates).	Lack mechanisms to distinguish between malicious traffic and unusual but legitimate behaviors (e.g., backups, software	Design ML models optimized for **low false positives** and interpretable

		updates).	scoring of threats, ensuring practical applicability.
Integration with SDN Controllers	IDS/IPS often operate as external modules or alerting systems, loosely connected to the SDN controller.	Limited real-time enforcement; alerts are not seamlessly translated into adaptive control actions.	Embed the threat evaluation module directly within the **SDN controller (e.g., Ryu)** for real-time scoring and seamless communication with the Adaptive Policy Engine.
Adaptive Security Response	Existing IDS solutions mostly provide binary alerts without enabling proportionate responses.	Lack of connection between detection results and adaptive policy enforcement mechanisms in SDN.	Enable **value-based outputs** (threat scores) that allow adaptive engines to choose responses like throttling, rerouting, quarantining, or dropping traffic.

RESEARCH PROBLEM

Software-Defined Networking (SDN) has become a revolutionary concept in the contemporary networking field by decoupling the data plane and control plane as well as providing centralized programmability. Although such an architectural change adds flexibility, scalability and dynamic policy enforcement, it also creates new vulnerabilities. The centralized SDN controller that is the decision-making organ comes an appealing target of adversaries, and in case malicious traffic is introduced into the network, it can spread quickly unless it is observed and countermeasures are taken in real time.

Despite the considerable progress made by previous studies in the field of intrusion detection of SDN setups, their primary emphasis has been on binary labeling of network traffic, i. e. as either benign or malicious. Such a dichotomous view is overly simplistic since cyber threats are not a uniform set of attacks that can be classified by strength, duration, and potential harm. The existing models are therefore unable to offer threat prioritization or contextual assessment thereby restricting the capacity of the controller to implement relative and reactive countermeasures. Say a brute-force login attempt and a volumetric DDoS attack are both considered malicious but the severity of either method and the urgency and response measures vary dramatically.

Moreover, current systems have false positives and false negatives, meaning that sometimes normal but suspicious traffic is mistaken to be malicious and, as a result, stops the service, whereas advanced threats could easily avoid detection. Lack of a threat evaluation mechanism that would give individual packets or flows a severity value or risk score leads to non-adaptive, rigorous security enforcement. This threatens one of the most likely benefits of SDN, namely, its capability to apply policies in a dynamic manner depending on the network situation.

Thus, the fundamental research issue addressed in this paper is the absence of a subtle, value-driven threat assessment system in SDN landscape. In the absence of such a mechanism, SDN controllers are limited to binary decisions, limiting the usefulness of adaptive policy engines and exposing networks to over-blocking of valid traffic and to under-responsiveness to intense attacks. To solve this issue, a module is needed that does not just examine features at the packet-level but also gives interpretable threat values that can be used to respond proportionately and automatically to the threat context.

OBJECTIVES

Main Objectives

The primary objective of this research is to design and implement a Threat Evaluation Component within the Software-Defined Networking (SDN) control plane that can intelligently inspect data packets and flow parameters, identify potential security threats, and assign a dynamic threat value reflecting their severity. This value-based evaluation aims to move beyond traditional binary intrusion detection approaches and provide actionable intelligence for adaptive and context-aware policy enforcement in SDN environments.

Specific Objectives

To achieve the above main objective, the study will focus on the following specific objectives:

1. To review and analyze existing SDN-based security mechanisms and intrusion detection approaches in order to identify limitations in binary classification and the lack of threat severity assessment.

2. To design a threat scoring framework that extracts relevant flow-level and packet-level features from the SDN controller and uses these features to compute a severity-oriented threat value.

3. To develop and integrate machine learning–based models trained on benchmark datasets such as CICIDS2018 with the aim of detecting anomalies and generating interpretable threat scores while minimizing false positives.

4. To implement a prototype of the proposed Threat Evaluation Component within an SDN controller (Ryu) and ensure its ability to function in real time without imposing significant overhead on the controller.

5. To evaluate the performance of the proposed component using both security metrics (accuracy, precision, recall, F1-score, false positive rate) and system-level metrics (latency, throughput, scalability), ensuring measurable improvements over existing solutions.

6. To validate the contribution of continuous threat scoring to adaptive SDN security, demonstrating how the proposed approach enhances resilience, enables proportional policy responses, and advances the state of the art in SDN-based intrusion detection.

METHODOLOGY

Overall System Description

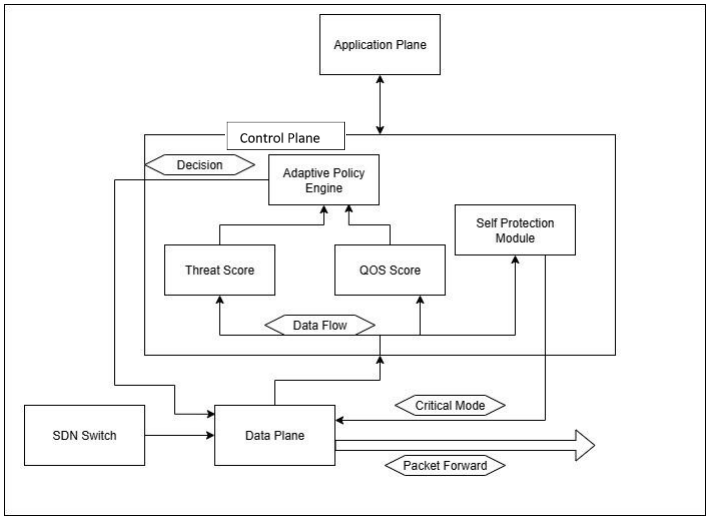


Figure0.1:Overall system Diagram

The proposed SDN framework consists of four key modules that collaboratively provide an adaptive, real-time threat detection and response framework. These modules are designed to work in sequence within the SDN architecture, offering both quality-of-service (QoS) assessment and threat detection and mitigation

1. Qos Scoring Module

The QoS Scoring Module is responsible for assessing the Quality of Service of incoming network traffic. This module analyzes flow-based features and assigns a QoS score to each data flow based on performance metrics such as latency, jitter, packet loss, and bandwidth usage.

2. Threat scoring module

The Threat Scoring Module is designed to detect potential threats in the incoming network traffic and assign a threat score based on the severity of the threat. This module performs anomaly detection and signature matching to identify abnormal traffic patterns or known attack signatures (e.g., DDoS, botnet traffic).

3. Adaptive policy engine

The Adaptive Policy Engine determines the appropriate action based on the threat score and QoS score of each network flow. It is the decision-making system that translates the scores into real-time network policies to mitigate threats or ensure service quality.

4. Critical Mode and System Response

The Critical Mode is an emergency response mechanism that is triggered when the system identifies high severity attacks (i.e. large scale DDoS attacks and zero-day exploits). The SDN system in this mode provides additional protection to curb damage.

Component Diagram

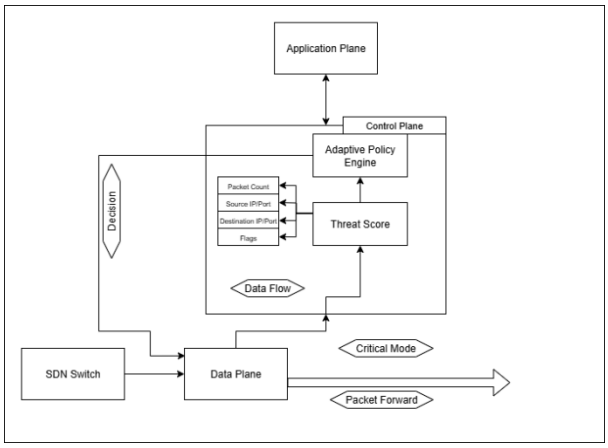


Figure2:Component Diagram

The research adopts a practical and experimental approach to developing a Threat Evaluation Component for Software-Defined Networking (SDN). The project will be divided into several stages: designing the components, integrating them within the SDN framework, and evaluating their performance. The project will utilize a combination of machine learning, network analysis, and SDN programming to achieve its objectives.

Data collection

- Select relevant network traffic datasets (e.g., CICIDS2018) for training and testing the models.
- Perform data preprocessing, including feature extraction, normalization, and handling missing data to ensure the datasets are suitable for machine learning analysis.

Feature extraction and model design

- Extract network flow features from incoming traffic such as packet counts, byte counts, protocol types, and flow duration.
- Design and implement machine learning models for anomaly detection and threat classification.
- Evaluate the model's performance using cross-validation and hyperparameter tuning to optimize detection accuracy and minimize false positives.

Threat scoring and evaluation

- Develop a dynamic threat scoring system that outputs a threat score (0 to 10 scale) for each flow based on the detected anomaly.
- Create an evaluation framework to measure the effectiveness of the threat scoring system based on metrics such as accuracy, precision, recall, and F1-score.

Sdn integration

- Integrate the Threat Evaluation Component into an SDN controller (e.g., Ryu), ensuring that it inspects and evaluates network traffic in real-time.
- Ensure seamless communication between the Threat Evaluation Module and the Adaptive Policy Engine for decision-making based on threat scores.

Testing and evaluation

- Simulate network traffic in an SDN environment using Mininet, introducing attack scenarios such as DDoS, brute-force attacks, and botnet communication.
- Evaluate the latency, throughput, and resource consumption of the system in real-time traffic conditions.

Critical mode implementation

- Implement a Critical Mode that triggers when high-severity threats are detected, limiting network access to pre-identified safe flows.
- Test Critical Mode functionality by simulating severe attack scenarios and assessing how the system behaves under these conditions.

3. Required data and collection method

- The data needed for this research includes network traffic datasets (CICIDS2018), which will be used to train, validate, and evaluate the machine learning models.
- The data will be collected from publicly available repositories and preprocessed to handle missing values and imbalanced data.
- Flow data will be extracted from the SDN controller to simulate real-world scenarios in an SDN environment.

4. Materials and resources needed

- **Software**:
    - SDN Controllers: Ryu or ONOS for integration.
    - Network Simulation Tools: Mininet for simulating the SDN network environment.
    - Dataset Repositories: Access to CICIDS2018
- **Hardware**:
    - High-performance computing resources for model training, particularly with deep learning models like LSTM.
    - SDN testbed environment with virtual machines or physical devices to deploy the SDN controller and evaluate the system.

5. Time frame and project schedule

A timeline of 12 months is proposed to complete the project. This includes time for the design and development of the system, as well as evaluation and testing.

Phase 1 (months 1-3):

- Data collection and preprocessing.
- Initial design and testing of the Threat Detection Model.

Phase 2 (months 4-6):

- Development of the Threat Scoring System and integration into SDN controllers.
- Initial testing of the model with simulated network traffic.

Phase 3 (months 7-9):

- Integration with the Adaptive Policy Engine.
- Implementation of the Critical Mode.

Phase 4 (months 10-12):

- Full system testing with attack simulations.
- Evaluation and fine-tuning of system performance.

6. Conclusion

This methodology provides a detailed, step-by-step process to develop an intelligent SDN security framework. The proposed system integrates anomaly-based detection with dynamic threat scoring and adaptive policy enforcement, ensuring a scalable and resilient solution that addresses the evolving security needs of SDN environments.

PROJECT REQUIREMENTS

1. Functional requirements

These requirements describe the core **functionalities** the system must support, and the tasks it must be able to perform to meet the objectives of the research:

1. **Data flow inspection:**
    - The system must be able to inspect incoming network flows in real time, analyzing various flow parameters (e.g., packet count, byte count, flow duration, source/destination IP, protocol type).
2. **Threat scoring:**
    - The system must compute a threat score for each data flow, assigning a value that reflects the severity of the detected threat. This score should range from 0 (benign) to 10 (high threat), based on anomaly detection and flow behavior analysis.
3. **Adaptive response mechanism:**

- The system should integrate with an adaptive policy engine, which will take the threat score and decide the action to be taken on the flow (e.g., allow, throttle, block, reroute).

4. **Critical mode activation:**
    - The system must include a Critical Mode that gets triggered in response to high-severity threats, restricting the network to only allow pre-identified safe flows.
5. **Real-time data processing:**
    - The system must process incoming flows with minimal latency, providing real-time threat detection and network control.
6. **Report generation:**
    - The system should be able to generate detailed security reports and performance logs that include threat scores, detected anomalies, and the actions taken.

2. Non-functional requirements

These requirements describe the performance characteristics of the system. These focus on the quality attributes rather than the core functions:

1. **Scalability:**
    - The system should be able to scale and handle large networks and high-throughput scenarios. It should handle increased traffic without a significant drop in performance.
2. **Reliability and availability:**
    - The system should operate reliably and should not experience downtime. It must be capable of handling network interruptions or controller failures without compromising security.
3. **Performance:**
    - The system must ensure low latency in processing data flows and generating threat scores. The processing time should be minimal to avoid disruptions in network traffic and performance.
4. **Security:**

- The system must ensure that sensitive data (e.g., flow data, threat scores) is securely handled, stored, and transmitted. Encryption and authentication measures must be in place.

5. **Usability:**
    - The system's interface must be user-friendly, especially for administrators who interact with the SDN controller. It should provide clear visualizations of threat data and system status.
6. **Maintainability:**
    - The system should be easy to maintain and upgrade. The modular architecture should support updates to the threat detection algorithms and network policies without major disruptions.

3. User requirements

These requirements focus on the end users of the system, such as network administrators, who will interact with the system. It defines what the users need from the system:

1. **User interface:**
    - Administrators should be able to easily configure, monitor, and manage network flows and policies through an intuitive web-based dashboard.
2. **Customizable security policies:**
    - Users should be able to define and adjust security policies in response to network needs and observed threat patterns.
3. **Alerting and notifications:**
    - The system should send alerts to administrators when threats are detected, particularly in critical scenarios. Alerts should include information on the threat score, affected flows, and actions taken.
4. **Report access**:
    - Administrators should be able to view and download security reports detailing threat analysis, actions taken, and system performance over time.

4. System requirements

These are the technical specifications needed for the system to function effectively:

1. **Sdn controller**:
    - The system must be compatible with popular SDN controllers such as Ryu or ONOS to manage and control network traffic.
2. **Machine learning libraries**:
    - The system will rely on machine learning frameworks such as Scikit-learn, TensorFlow, or Keras for threat detection and scoring.
3. **Simulation tools**:
    - Mininet will be used for network simulation, enabling the testing of real-world traffic scenarios and attacks in a virtualized environment.
4. **Data storage**:
    - The system will require a database (e.g., MySQL or MongoDB) to store flow data, threat scores, and historical reports for analysis and troubleshooting.

5. Use cases (tentative)

1. **Use case 1**: *Detecting and Mitigating a DDoS Attack*
    - When a high volume of traffic is detected from a single source, the system classifies the flow as a potential DDoS attack and assigns a high threat score. The Adaptive Policy Engine responds by throttling or blocking the malicious traffic.
2. **Use case 2**: *Flow Inspection and Threat Detection*
    - The system scans the regular network flows, comparing whether a flow is misbehaving. In case of an anomaly detection (e.g., port scan or unauthorized access attempt) the system labels the threat with a threat score according to the severity of the detected attack.

6. Test cases (tentative)

1. **Test case 1**: *Accuracy of Threat Scoring*
    - **Objective**: Ensure the system generates accurate threat scores that match predefined attack scenarios.

- **Expected outcome**: The system should classify traffic accurately and assign appropriate threat values.

2. **Test case 2**: *System Response in Critical Mode*
    - **Objective**: Test the behavior of the system when a **critical threat** (e.g., zero-day attack) is detected.
    - **Expected Outcome**: The system should enter **Critical Mode** and block all untrusted traffic while allowing previously safe flows.
3. **Test case 3**: *Performance Under Load*
    - **Objective**: Evaluate the system's performance under high traffic volume.
    - **Expected outcome**: The system should process a high number of flows with **minimal latency**.

7. Wireframes (tentative)

Wireframes for the system interface, such as the **user dashboard**, should include:

- **Flow visualizations** showing the threat status of individual network flows.
- **Alerts and notifications** regarding critical events and actions.
- **Report download options** to allow users to access threat reports in PDF or other formats.

REFERENCES

1. D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky and S. Uhlig, "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14-76, Jan. 2015, doi: 10.1109/JPROC.2014.2371999.

2. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," *4th International Conference on Information Systems Security and Privacy (ICISSP)*, Funchal, Madeira, Portugal, 2018, pp. 108–116, doi: 10.5220/0006639801080116.

3. A. Moubayed, A. Refaey, H. Shami, and A. Shami, "Software Defined Perimeter (SDP): State of the Art Secure Solution for Modern Networks," *IEEE Network*, vol. 33, no. 5, pp. 226-233, Sept.–Oct. 2019, doi: 10.1109/MNET.001.1800523.

4. P. Idhammad, K. Afdel, and M. Belouch, "Detection System of HTTP DDoS Attacks in a Cloud Environment Based on Information Theoretic Entropy and Random Forest," *Security and Communication Networks*, vol. 2018, Article ID 1263123, pp. 1–13, 2018, doi: 10.1155/2018/1263123.